

# Теория формальных языков

Клышинский Э.С.

2026 г.

# Рекомендуемая литература

- Ахо, Лам, Сети, Ульман: Компиляторы. Принципы, технологии и инструментарий, Вильямс, 2019 г., 1184 с.
- Фридл Регулярные выражения. O'Reilly, 2018 г., 608 с.
- А. Е. Пентус, М. Р. Пентус Теория формальных языков, 2004 г. 80 с.

# Лекция 1

Символы, цепочки,  
грамматики

# Определения

## **Язык (оставим пока без определения)**

- естественный: русский, английский, ...;
- искусственный: эсперанто, ...;
- ограниченный естественный: диалог с банковским чат-ботом;
- формальные: Python, HTML, PDF, VHDL, UML.

# Определения

## Алфавит (множество терминалов)

$V_T$  – это непустое конечное множество СИМВОЛОВ.

$V_{T1} = \{a, b, c\}$

$V_{T2} = \{if, var, const, <, >, =, ==, !=, :, \varepsilon\}$

$\varepsilon$  – пустой символ или е-цепочка.

# Определения

Всякая конечная последовательность символов алфавита  $A$  называется **цепочкой**.

Цепочки VT1 :  $a, ab, bb, abbcca$

Цепочки VT2 :  $if\ var > var:$   
 $if\ var \geq const:$

# Определения

$$a \cdot b = ab$$

$$ab \neq ba$$

$$a \text{ OR } b = a|b$$

$$x^0 = \varepsilon, x^1 = x, x^2 = xx, \dots$$

$$x^n = xx \dots x \Rightarrow x^n = x^{n-1}x = xx^{n-1}$$

# Итерации

**Полная итерация** обозначается как  $x^*$  и означает произвольное повторение цепочки  $x$ .

**Положительная итерация** обозначается как  $x^+$  и означает произвольное ненулевое повторение цепочки  $x$ .



# Итерации - некоторые свойства

Итерации могут преобразовываться одна в другую.

$$X^+ = XX^* = X^*X.$$

$$X^* = \varepsilon | X^+ = X^+ | \varepsilon.$$

# Определения

**Продукцией** или **правилом подстановки** называется упорядоченная пара  $(U, x)$

записываемая как

$U ::= x,$

где  $U$  – некоторый символ (левая часть правила),

$x$  – цепочка символов (правая часть правила).

# Пример записи продукций

$\langle \text{DIGIT} \rangle ::= 0$

$\langle \text{DIGIT} \rangle ::= 1$

...

$\langle \text{DIGIT} \rangle ::= 9$

Или более коротко

$\langle \text{DIGIT} \rangle ::= 0 \mid 1 \mid \dots \mid 9$

# Пример записи продукций

Правая часть может состоять не только из терминалов, но и нетерминалов.

$$\langle \text{NUMBER} \rangle ::= \langle \text{DIGIT} \rangle$$
$$\langle \text{NUMBER} \rangle ::= \langle \text{DIGIT} \rangle \langle \text{NUMBER} \rangle$$

Или в другой записи

$$\langle \text{NUMBER} \rangle ::= 0 \mid 1 \mid \dots \mid 9 \mid$$
$$0 \langle \text{NUMBER} \rangle \mid 1 \langle \text{NUMBER} \rangle \mid \dots \mid 9 \langle \text{NUMBER} \rangle$$

# Немного о формате записи

Подобный формат называется БНФ –  
Бэкусовские нормальная форма или  
Бэкуса-Науэра форма  
(калька с Backus Normal Form - BNF).

# (Почти) Определения

Для данной грамматики `<NUMBER>` и `<DIGIT>` будут называться **нетерминалами**, а символы `0, 1, ..., 9` – **терминалами**.  
В некотором смысле, терминалы - это то, из чего состоят цепочки языка, а нетерминалы - названия правил. (Хотя на самом деле это совсем не так.)

# (Почти) Определения

В данной грамматике `<NUMBER>` будет называться начальным символом, так как разбор начинается именно с него. Хотя позже мы увидим, что им разбор может и заканчиваться.

# Определения

**Грамматикой** называется четверка  
 $G = \{VN, VT, P, E\}$ .

$VN$  – множество нетерминалов,

$VT$  – множество терминалов,

$P$  – набор правил, описывающих цепочки,  
принадлежащие описываемому языку и

$E$  – начальный символ грамматики.



# САМОЕ Важное Свойство

Левая и правая части эквивалентны.  
Это означает, что в цепочках можно  
проводить замены левой части на правую  
или наоборот.

# Определения

Цепочка  $V$  **непосредственно порождает** цепочку  $W$  и обозначают это как  $V \Rightarrow W$ , если  $V = xUy$ ,  $W = xuy$  и  $\exists U ::= u$ .

$x$  и  $y$  – любые (в том числе пустые) цепочки символов.

Цепочка  $V$  **порождает**  $W$  ( $W$  выводима из  $V$ ) и обозначают это как  $V \Rightarrow^* W$ , если  $\exists V \Rightarrow V' \Rightarrow V'' \Rightarrow \dots \Rightarrow W$ .

# Определения

Если  $V$  является начальным символом грамматики  $G$ , то говорят, что

$W$  **выводима из этой грамматики.**

При этом  $W$  называется **сентенциальной формой**. **Предложением** называется сентенциальная форма, состоящая только из терминальных символов.

**Язык** – это множество предложений, выводимых из данной грамматики.

# Пример грамматики

**НЕТЕРМИНАЛЫ, описанные на слайде**  
**НЕТЕРМИНАЛЫ, отсутствующие на слайде**

**Терминалы**

**Фрагменты БНФ**

*(Это не какие-то принятые цвета, это  
мне захотелось разрисовать для  
наглядности)*

# Пример грамматики

**HTML** ::= **HEAD** **BODY**

**HEAD** ::= **<head>** **TITLE** **HEAD2** **</head>**

**HEAD2** ::= **STYLE** | **SCRIPT** |  
                  **STYLE** **HEAD2** |  
                  **SCRIPT** **HEAD2**

**STYLE** ::= **<style>** **STYLE2** **</style>**

**STYLE2** ::= **STYLE\_LINE** |  
                  **STYLE\_LINE** **STYLE2**

**STYLE\_LINE** ::= **.STYLE\_ID** {**STYLES**}

**STYLE\_ID** ::= **CHAR** | **CHAR** **STYLE\_ID**

# Разбор с помощью грамматик

Пусть дана грамматика

$$\langle E \rangle ::= \langle T \rangle \mid \langle E \rangle + \langle E \rangle$$
$$\langle T \rangle ::= (\langle E \rangle) \mid id$$

И цепочка

$$id + id + id$$

# Разбор с помощью грамматик

Пусть дана грамматика

$$\langle E \rangle ::= \langle T \rangle \mid \langle E \rangle + \langle E \rangle$$
$$\langle T \rangle ::= (\langle E \rangle) \mid id$$

**Нисходящий вывод предложения**

$$\begin{aligned} E &\rightarrow E + E \rightarrow E + E + E \rightarrow T + E + E \rightarrow \\ id + E + E &\rightarrow id + T + E \rightarrow id + id + E \rightarrow \\ id + id + T &\rightarrow id + id + id \end{aligned}$$

# Разбор с помощью грамматик

Пусть дана грамматика

$$E ::= \langle T \rangle \mid \langle E \rangle + \langle E \rangle$$
$$T ::= (\langle E \rangle) \mid id$$

**Восходящий вывод предложения**

$$\begin{aligned} id + id + id &\rightarrow id + id + T \rightarrow id + id + E \rightarrow \\ id + T + E &\rightarrow id + E + E \rightarrow T + E + E \rightarrow \\ E + E + E &\rightarrow E + E \rightarrow E \end{aligned}$$



# Разбор с помощью грамматик

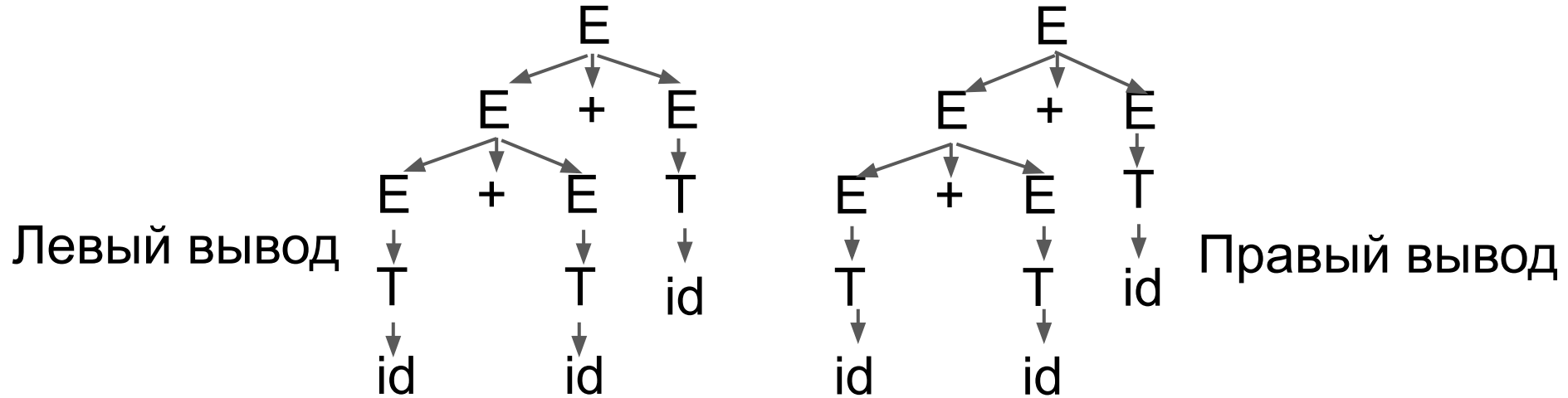
Пусть дана грамматика

$$E ::= id \mid \langle E \rangle - \langle E \rangle \quad | \quad (\langle E \rangle)$$

Восходящий вывод предложения

$$\begin{aligned} id - (id - id) &\rightarrow id - (id - E) \rightarrow \\ id - (E - E) &\rightarrow E - (E - E) \rightarrow E - (E) \rightarrow \\ E - E &\rightarrow E \end{aligned}$$

Пусть дана грамматика и цепочка  
 $E ::= \langle T \rangle \mid \langle E \rangle + \langle E \rangle$   $\text{id} + \text{id} + \text{id}$   
 $T ::= (\langle E \rangle) \mid \text{id}$



Деревья разбора

Это был нисходящий вывод.

# Эквивалентные грамматики

A ::= B C  
B ::= while ( G )  
C ::= D | E  
D ::= exp  
E ::= { F }  
F ::=  $\varepsilon$  | exp F  
G ::= var H var  
H ::= < | > | <= | >= |  
      == | !=

WH ::= WH2 OP  
WH2 ::= while ( L\_EXPR )  
OP ::= EXP | BR\_EXP  
EXP ::= exp  
BR\_EXP ::= { M\_EXP }  
M\_EXP ::=  $\varepsilon$  | exp M\_EXP  
L\_EXPR ::= var LOP var  
LOP ::= < | > | <= | >= |  
      == | !=

# Эквивалентные грамматики

$A ::= 0B | 1C$

$B ::= 11$

$C ::= 0C | 0$

$A ::= 0B | 1C$

$B ::= 1B'$

$B' ::= 1D$

$C ::= 0C | 0C'$

$C' ::= \varepsilon$

$D ::= \varepsilon$

$A ::= \varepsilon$

$B ::= A0$

$B' ::= B1$

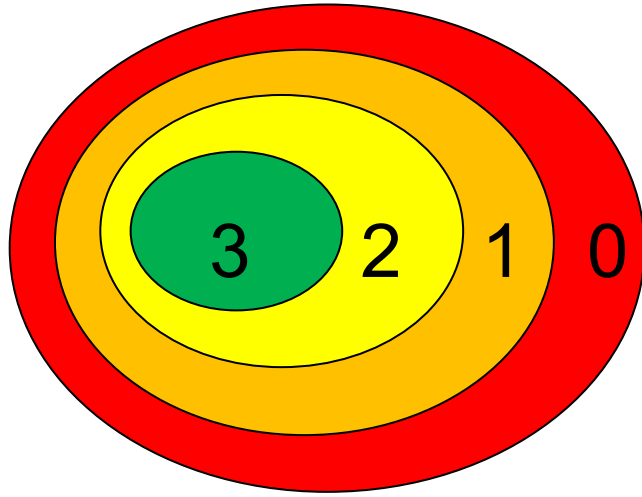
$C ::= A1$

$C' ::= C0 | C'0$

$D ::= B'1$

# Классификация грамматик

# Иерархия грамматик по Хомскому



**Тип 3** (регулярные)

$A ::= a\alpha$ , где  $\alpha \in VN$  или  $\alpha = \varepsilon$ ,  
 $a \in VT$ ,  $A \in VN$

**Тип 2** (контекстно-свободные)

$A ::= \alpha\beta$ , где  $\alpha$  и  $\beta \in \{VT, VN\}^*$

**Тип 1** (контекстно-зависимые)

$\alpha A \beta ::= \alpha \gamma \beta$ , где  $A \in VN$ ,  
 $\alpha$  и  $\beta \in \{VT, VN\}^*$ ,  $\gamma \in \{VT, VN\}^+$

**Тип 0** (неограниченные)  $\alpha ::= \beta$ , где  
 $\alpha \in \{VT, VN\}^+$  и содержит хотя бы  
один нетерминал,  $\beta \in \{VT, VN\}^*$

# Регулярные грамматики

$A ::= a\alpha$ , где  $\alpha \in VN$  или  $\alpha = \varepsilon$ ,  $a \in VT$ ,  $A \in VN$

Самый простой вид грамматик –  
как для написания, так и с точки зрения  
языка.

Запись при помощи более удобных  
регулярных выражений вместо грамматик.  
Скорость разбора строк – линейна.

Грамматика для языка  $a^n$ :

$A ::= a \mid aA$

# Контекстно-свободные грамматики

$A ::= \alpha\beta$ , где  $\alpha$  и  $\beta \in \{V_T, V_N\}^*$

Простой вид грамматик – человеку обычно удобнее писать КС-грамматики, чем соблюдать регулярность.

Время разбора в худшем случае пропорциональна длине строки на число продукций.

Грамматика для языка  $a^n b^n$  :

$A ::= ab \mid aAb$



# Контекстно-зависимые грамматики

$\alpha A \beta ::= \alpha \gamma \beta$ , где  $A \in VN$ ,  
 $\alpha$  и  $\beta \in \{VT, VN\}^*$ ,  $\gamma \in \{VT, VN\}^+$

Непростой вид грамматик.

Время разбора в худшем случае  
экспоненциально.

# Контекстно-зависимые грамматики

Грамматика для языка  $a^n b^n c^n$  :

$$S \rightarrow aSBC$$

$$S \rightarrow aBC$$

$$CB \rightarrow HB$$

$$HB \rightarrow HC$$

$$HC \rightarrow BC$$

$$aB \rightarrow ab$$

$$bB \rightarrow bb$$

$$bC \rightarrow bc$$

$$cC \rightarrow cc$$

# Неограниченные грамматики

$\alpha ::= \beta$ , где  $\alpha \in \{VT, VN\}^+$  и содержит хотя бы один нетерминал,  $\beta \in \{VT, VN\}^*$

*Практического применения в силу своей сложности такие грамматики не имеют.*

*Полны по Тьюрингу нормальный алгоритм Маркова, 2-теговая система, клеточный автомат с правилом 110, ингибиторная сеть Петри. Полными по Тьюрингу являются также неограниченные грамматики.*

Как писать грамматики

# Правила написания БНФ

Если в последовательности можно выделить несколько идущих подряд более простых подпоследовательностей, их следует оформлять как одну продукцию с несколькими идущими подряд символами:

$A ::= BC.$

# Правила написания БНФ

Если в последовательности имеется несколько альтернативных вариантов разбора, необходимо оформить их как различные продукции одного и того же правила:

$$A ::= V \mid C.$$

# Правила написания БНФ

$a^* \Rightarrow A ::= aA \mid \varepsilon$  или  $A ::= Aa \mid \varepsilon$

$a^+ \Rightarrow A ::= aA \mid a$  или  $A ::= Aa \mid a$

Или в более общем случае,

$a^*b \Rightarrow A ::= aA \mid b$  или  $A ::= Vb, V ::= \varepsilon \mid Va;$

$ba^* \Rightarrow A ::= Aa \mid b$  или  $A ::= bV, V ::= \varepsilon \mid aV.$

При  $b = \varepsilon$  получим  $a^*$ , а при  $b = a$  получим  $a^+$ .

# Правила написания БНФ

KISS – keep it simple, stupid



# Неоднозначные грамматики

**Грамматика, для которой существует цепочка для которой можно построить более одного дерева вывода, называется неоднозначной.**

В общем случае доказать, что грамматика является однозначной, невозможно.

# Неоднозначные грамматики

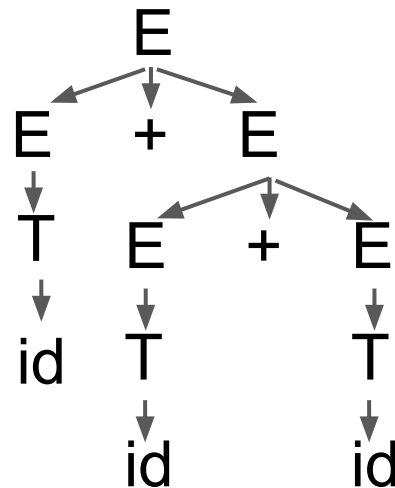
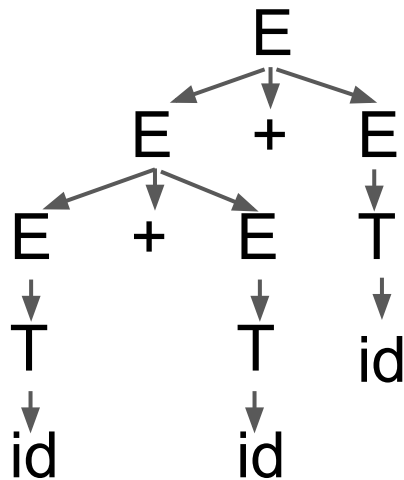
Пусть дана грамматика

$E ::= \langle T \rangle \mid \langle E \rangle + \langle E \rangle$

$T ::= (\langle E \rangle) \mid id$

И цепочка

$id + id + id$



# Неоднозначные грамматики

$E ::= \langle T \rangle \mid \langle E \rangle + \langle E \rangle$  - неоднозначная

$T ::= (\langle E \rangle) \mid \text{id}$

$E ::= \langle T \rangle \mid \langle T \rangle + \langle E \rangle$  - однозначная

$T ::= (\langle E \rangle) \mid \text{id}$

# Метод разбора рекурсивным спуском

Пусть дана

грамматика

$$E ::= \langle T \rangle \mid \langle T \rangle \langle E2 \rangle$$
$$E2 ::= + \langle T \rangle \mid + \langle T \rangle \langle E2 \rangle$$
$$T ::= \langle F \rangle \mid \langle F \rangle \langle T2 \rangle$$
$$T2 ::= * \langle F \rangle \mid * \langle F \rangle \langle T2 \rangle$$
$$F ::= (\langle E \rangle) \mid \text{id}$$

# Метод разбора рекурсивным спуском

Пусть дана

грамматика

$E ::= \langle T \rangle \mid \langle T \rangle \langle E2 \rangle$

$E2 ::= + \langle T \rangle \mid + \langle T \rangle \langle E2 \rangle$

$T ::= \langle F \rangle \mid \langle F \rangle \langle T2 \rangle$

$T2 ::= * \langle F \rangle \mid * \langle F \rangle \langle T2 \rangle$

$F ::= (\langle E \rangle) \mid id$

```
def E():
```

```
    global cur_pos
```

```
    pos = cur_pos
```

```
    if not T():
```

```
        cur_pos = pos
```

```
        return False
```

```
    E2()
```

```
    return True
```

# Метод разбора рекурсивным спуском

Пусть дана  
грамматика

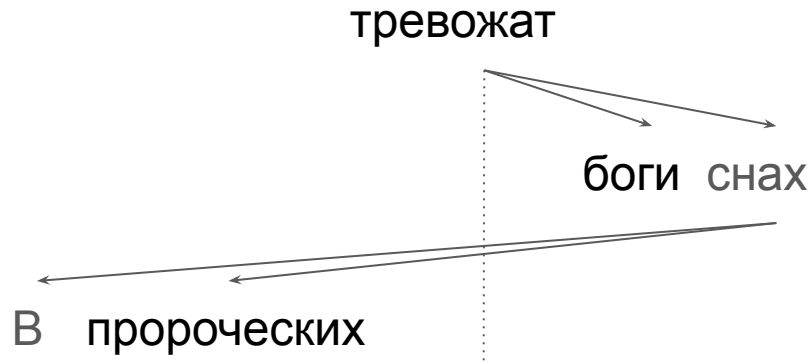
$$E ::= \langle T \rangle \mid \langle T \rangle \langle E2 \rangle$$
$$E2 ::= + \langle T \rangle \mid + \langle T \rangle \langle E2 \rangle$$
$$T ::= \langle F \rangle \mid \langle F \rangle \langle T2 \rangle$$
$$T2 ::= * \langle F \rangle \mid * \langle F \rangle \langle T2 \rangle$$
$$F ::= (\langle E \rangle) \mid \text{id}$$

```
def E2():  
    global cur_pos  
    pos = cur_pos  
    if is_lexeme("+"):  
        if not T():  
            cur_pos = pos  
            return False  
        E2()  
        return True  
    else:  
        return False
```

# Метод разбора рекурсивным спуском

1. Здесь неявно подразумевается, что в случае неуспеха позиция сама восстанавливается на последнюю успешно разобранную.
2. При переходе от грамматики к коду мы провели левую факторизацию грамматики.

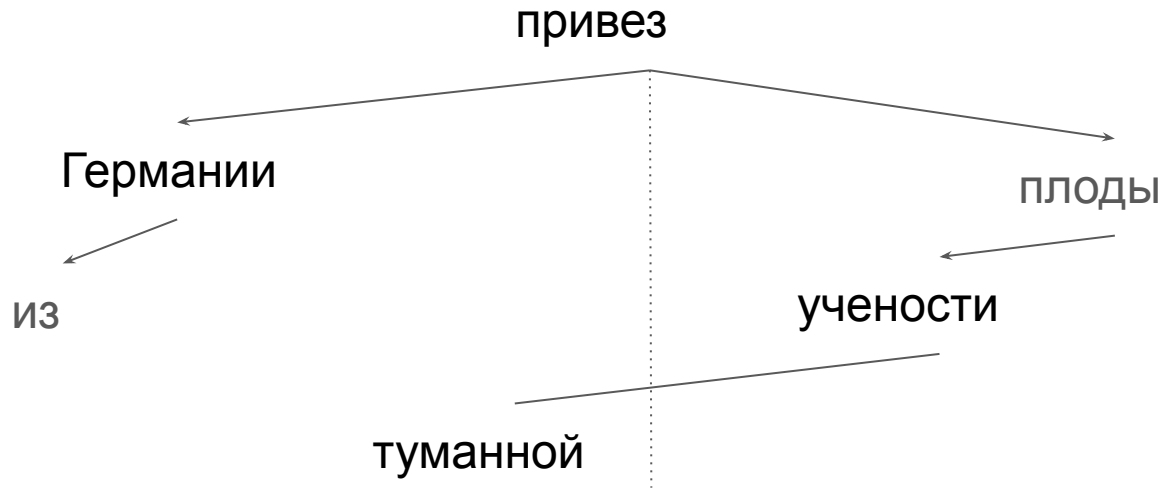
# Пример разбора



([В пророческих <тревожат> {боги} снах])

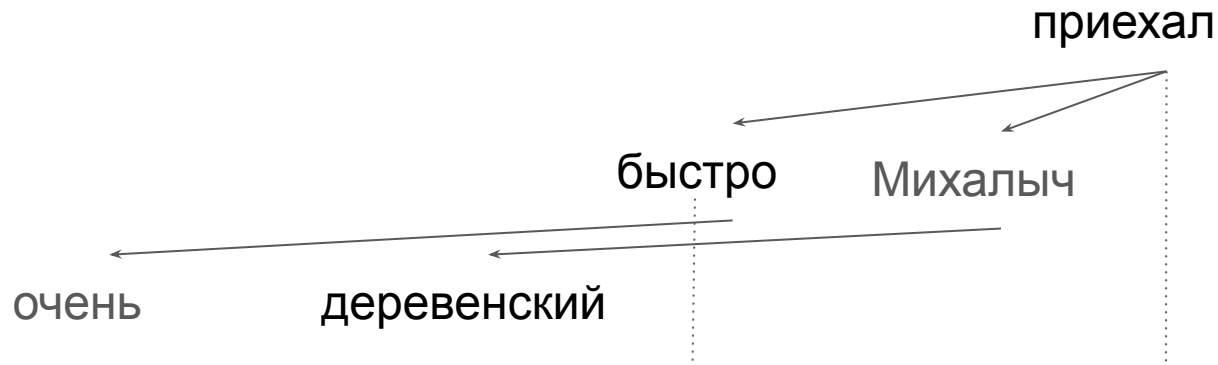


# Пример разбора



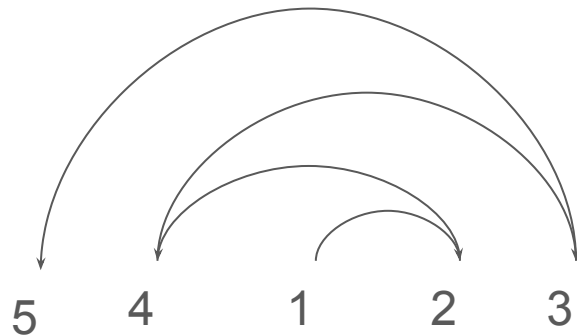
Он из Германии {<туманной (привез) учености> плоды}

# Пример разбора



Очень деревенский быстро Михалыч приехал

# Пример разбора



# LR(k)-грамматики

